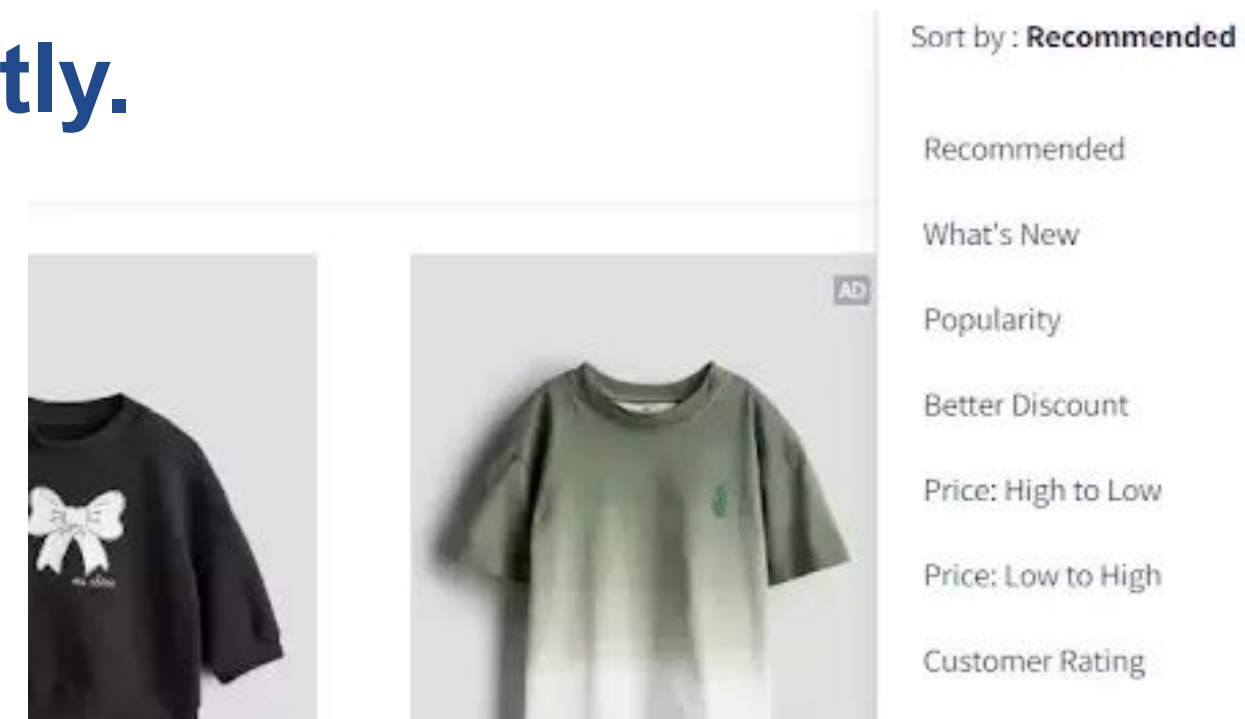
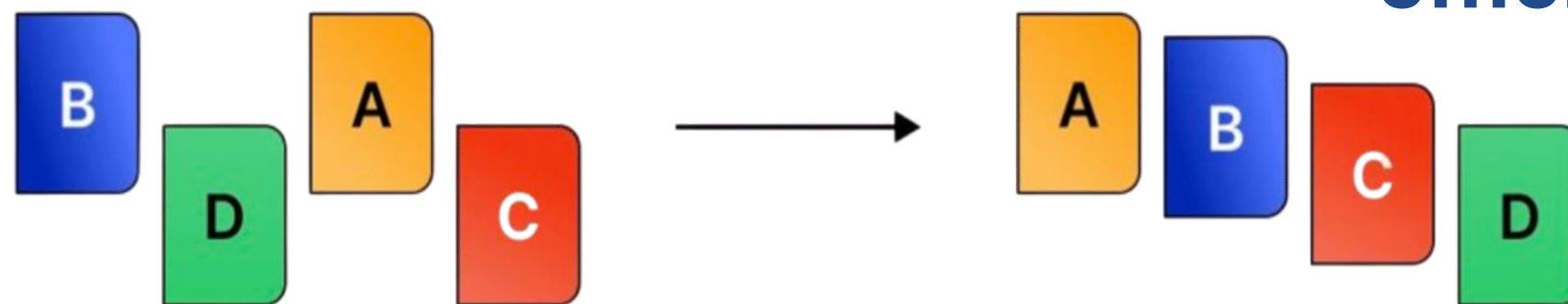


What are Sorting Algorithms 🤔?

Sorting algorithms in data structure are methods used to arrange data in a specific order, like ascending or descending.

Imagine you have a list of numbers or names, and you want to organize them from smallest to largest, or alphabetically. Sorting algorithms help you do that efficiently.



Classification of Sort Algorithms

Based on

Comparison:
Comparison-based Sorting: These algorithms sort data by comparing elements. Examples include Bubble

Sort, Selection Sort, Insertion Sort, Merge Sort, Quick Sort, and Heap Sort.
Non-comparison-based Sorting: These algorithms sort data without comparing elements directly.

Examples include Counting Sort, Radix Sort, and Bucket Sort.

In-Place & Not In-Place Sorting:

TYPE	DESCRIPTION	EXAMPLE
In-Place	Uses constant extra space	Quick Sort, Bubble Sort
Not In-Place	Requires additional memory	Merge Sort, Counting Sort

Adaptive & Non-Adaptive Sorting:

TYPE	BEHAVIOR	EXAMPLE
Adaptive	Takes advantage of existing order in input	Insertion Sort, Tim Sort
Non-Adaptive	Performs same operations regardless	Selection Sort, Merge Sort

Internal & External Sorting:

TYPE	WHEN USED	EXAMPLE
Internal	All data fits in RAM	Quick Sort, Merge Sort
External	Data too large for RAM (on disk)	Multi-way Merge Sort, External Merge Sort

Stable & Unstable Sorting:

TYPE	MAINTAINS RELATIVE ORDER OF EQUAL ELEMENTS?	EXAMPLE
Stable	Yes	Bubble Sort, Merge Sort, Counting Sort
Unstable	No	Quick Sort, Heap Sort, Selection Sort

In-Built sort() Function in C++: Arrays & Vectors

```

1 // ♦ Sorting an array in ascending order:
2 Sort(arr, arr + n);
3
4 // ♦ Sorting an array in descending order:
5 Sort(arr, arr + n, greater<int>());
6
7 // ♦ Sorting an array from startIndex to endIndex (inclusive):
8 Sort(arr + startIndex, arr + endIndex + 1);
9
10 // ♦ Sorting a vector in ascending order:
11 Sort(vec.begin(), vec.end());
12
13 // ♦ Sorting a vector in descending order:
14 Sort(vec.begin(), vec.end(), greater<int>());
15
16 // ♦ Sorting a vector from startIndex to endIndex (inclusive):
17 Sort(vec.begin() + startIndex, vec.begin() + endIndex + 1);

```

Algorithm	Time Complexity			Space Complexity
	Best	Average	Worst	Worst
<u>Quicksort</u>	$\Omega(n \log(n))$	$\Theta(n \log(n))$	$O(n^2)$	$O(\log(n))$
<u>Mergesort</u>	$\Omega(n \log(n))$	$\Theta(n \log(n))$	$O(n \log(n))$	$O(n)$
<u>Timsort</u>	$\Omega(n)$	$\Theta(n \log(n))$	$O(n \log(n))$	$O(n)$
<u>Heapsort</u>	$\Omega(n \log(n))$	$\Theta(n \log(n))$	$O(n \log(n))$	$O(1)$
<u>Bubble Sort</u>	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$
<u>Insertion Sort</u>	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$
<u>Selection Sort</u>	$\Omega(n^2)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$
<u>Tree Sort</u>	$\Omega(n \log(n))$	$\Theta(n \log(n))$	$O(n^2)$	$O(n)$
<u>Shell Sort</u>	$\Omega(n \log(n))$	$\Theta(n(\log(n))^2)$	$O(n(\log(n))^2)$	$O(1)$
<u>Bucket Sort</u>	$\Omega(n+k)$	$\Theta(n+k)$	$O(n^2)$	$O(n)$
<u>Radix Sort</u>	$\Omega(nk)$	$\Theta(nk)$	$O(nk)$	$O(n+k)$
<u>Counting Sort</u>	$\Omega(n+k)$	$\Theta(n+k)$	$O(n+k)$	$O(k)$
<u>Cubesort</u>	$\Omega(n)$	$\Theta(n \log(n))$	$O(n \log(n))$	$O(n)$



Problems to Solve

Problem 1: Grouping Anagrams Together

Identify and group anagrams from a list of words, then print each group together.

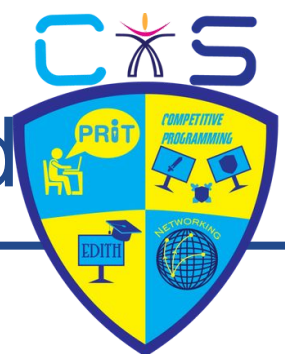
- Input: An array of strings (words).
- Output: Display grouped anagrams.

For example, given:

["care", "tap", "race", "apt", "acre"]

A valid output would be:

care race acre tap apt (“care,” “race,” and “acre” are anagrams, as are “tap” and
“apt.”)



Problem 2: Sort Array into Wave Form

You are tasked with rearranging an unsorted array of n integers into a wave-like structure. An array is considered to be in wave form if it adheres to the following

pattern: $A[0] \geq A[1] \leq A[2] \geq A[3] \leq A[4] \geq \dots$ and so forth.

Input: An array of integers

Output:

The same array rearranged into any valid wave form.

Example:

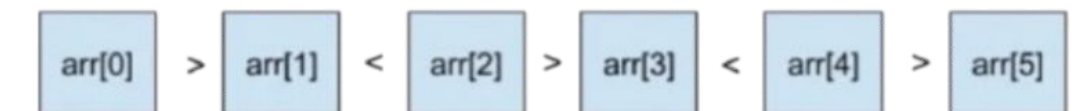
Input: $A = [1, 2, 3, 4]$

Output: $[2, 1, 4, 3]$ or $[4, 1, 3, 2]$ or any other valid wave pattern.

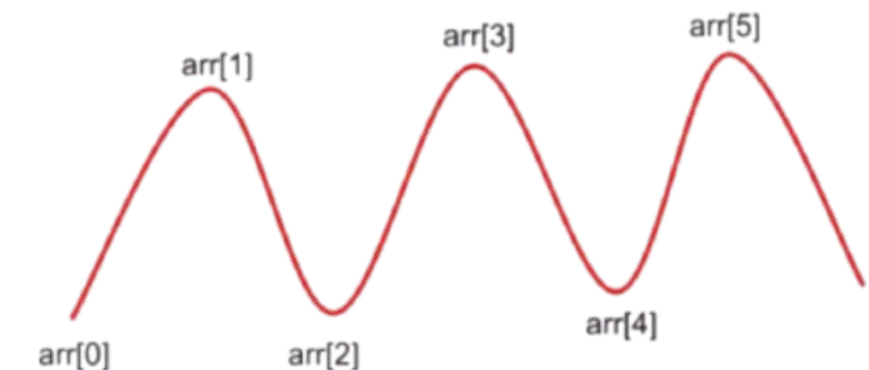
Input:



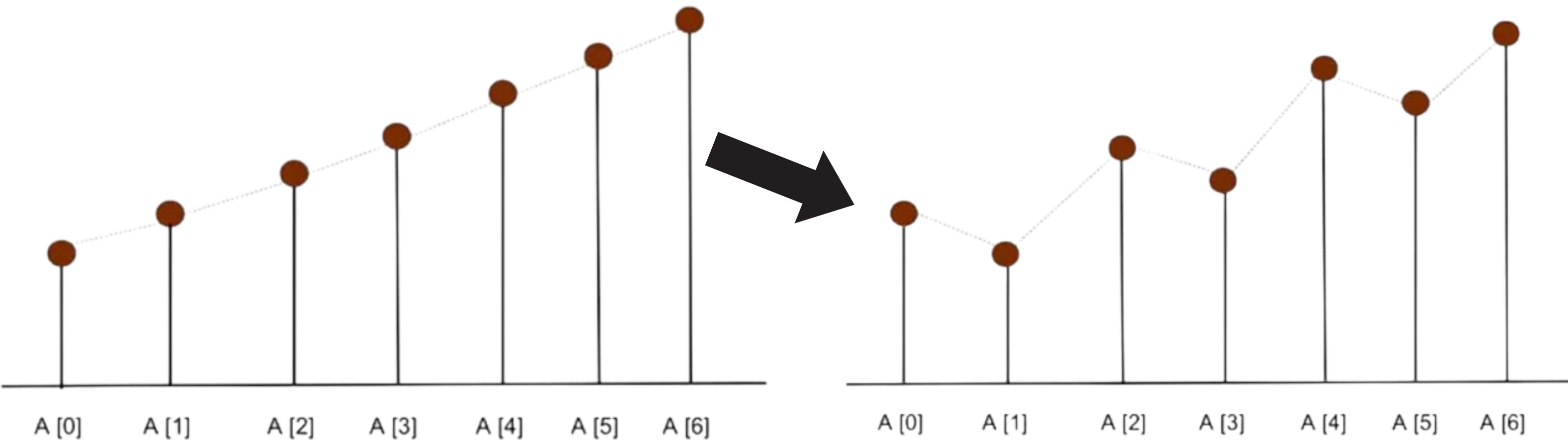
Output in array form:



Output in waveform:



One possible approach to finding a solution is



An ideal method of approach.

```
void wave_form(int a[], int n)
{
    // Traverse all even elements
    for (int i=0; i<n; i+=2)
    {
        // If current even element is smaller than previous
        if (i>0 && a[i-1] > a[i])
            swap(&a[i], &a[i-1]);

        // If current even element is smaller than next
        if (i<n-1 && a[i] < a[i+1])
            swap(&a[i], &a[i + 1]);
    }
}
```




What is Merge Sort?

Merge Sort is an efficient and stable divide-and-conquer algorithm utilized for sorting elements by:

- Dividing the list into halves recursively until each sublist contains only one element (base case).
- Conquering by merging these sublists back together in sorted order.

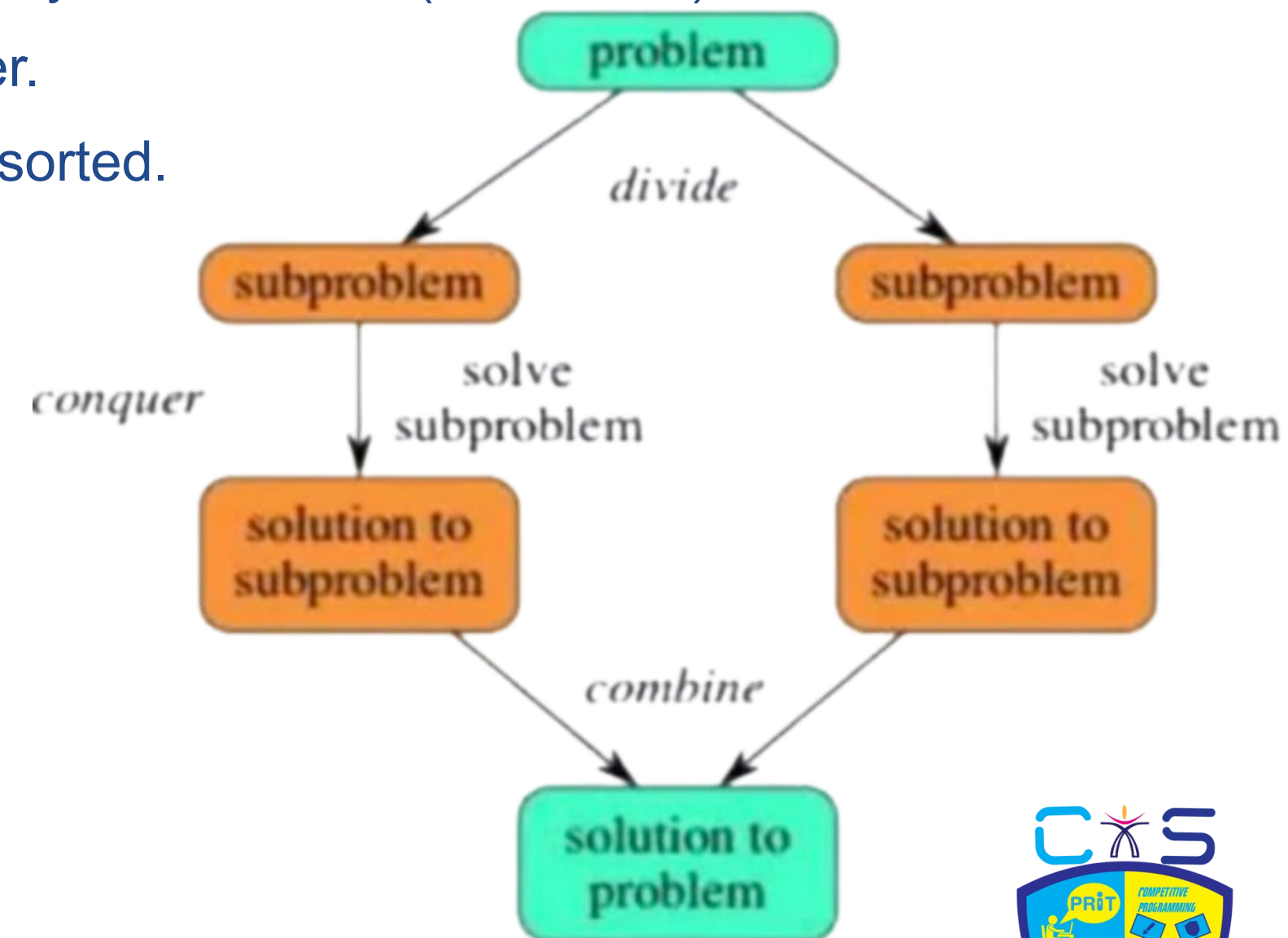
Each merge step results in larger sorted sublists until the entire list is sorted.

Key Features:

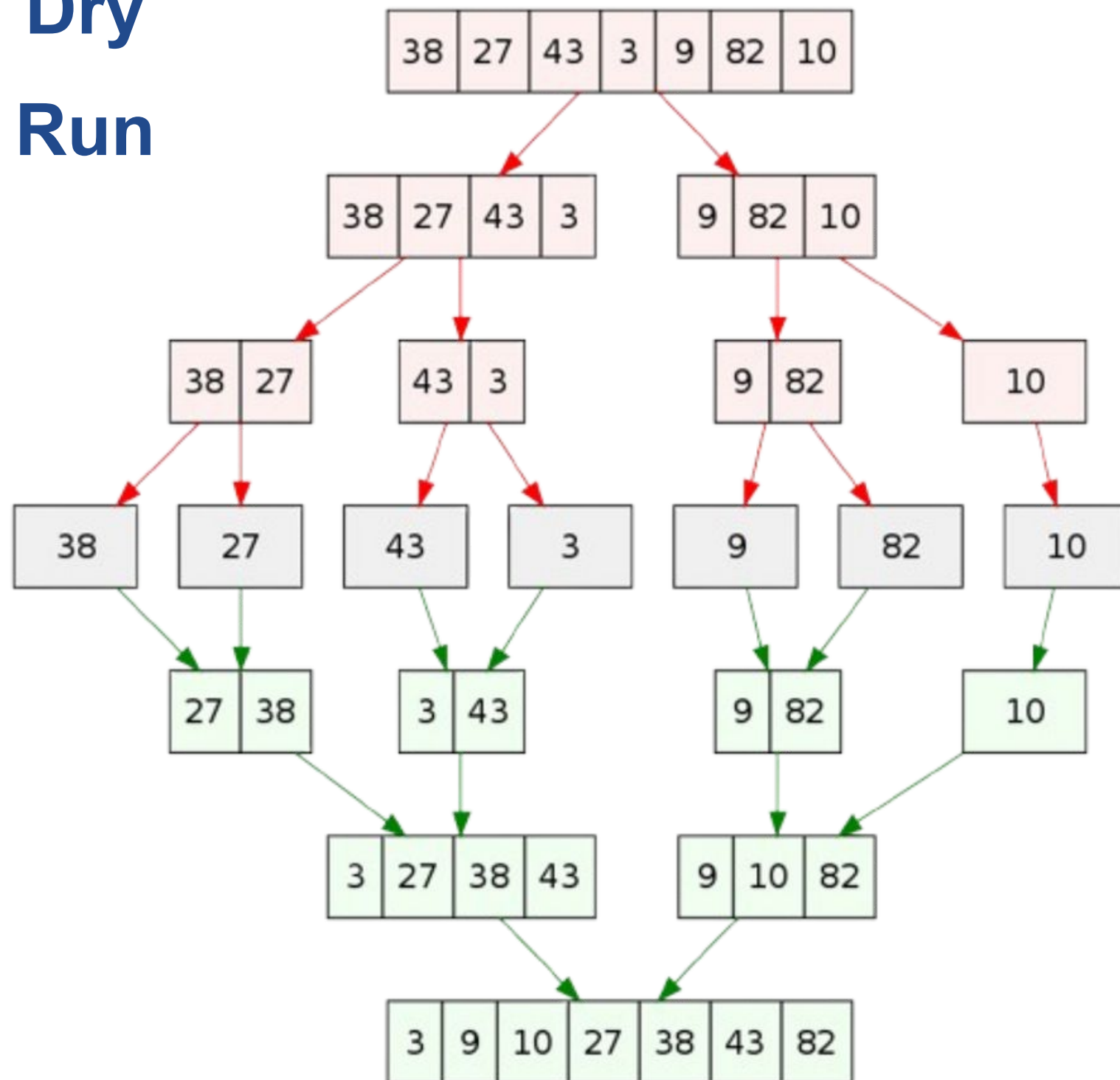
- Time Complexity:
 - Best, Average, Worst: $O(n \log n)$
- Space Complexity:
 - $O(n)$ (additional memory needed for merging)
- Stable Sort: ✓ Maintains the relative order of equal elements

Used When:

- Consistent performance is required
- Stability is essential
- Sorting linked lists or external data



Dry Run



```

1
2  #include <iostream>
3  using namespace std;
4
5  void merge(int arr[], int l, int m, int r) {
6      int n1 = m - l + 1, n2 = r - m;
7      int L[n1], R[n2];
8
9      for (int i = 0; i < n1; i++) L[i] = arr[l + i];
10     for (int i = 0; i < n2; i++) R[i] = arr[m + 1 + i];
11
12     int i = 0, j = 0, k = l;
13     while (i < n1 && j < n2)
14         arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
15     while (i < n1) arr[k++] = L[i++];
16     while (j < n2) arr[k++] = R[j++];
17 }
18
19 void mergeSort(int arr[], int l, int r) {
20     if (l < r) {
21         int m = (l + r) / 2;
22         mergeSort(arr, l, m);
23         mergeSort(arr, m + 1, r);
24         merge(arr, l, m, r);
25     }
26 }
27
28 int main() {
29     int arr[] = {5, 2, 8, 4, 1}, n = 5;
30     mergeSort(arr, 0, n - 1);
31     for (int i = 0; i < n; i++) cout << arr[i] << " ";
32 }
  
```


Given an integer array `arr[]` of size `n`, find the inversion count in the array.

Two array elements `arr[i]` and `arr[j]` form an inversion if `arr[i] > arr[j]` and `i < j`.

Input: `arr() = {4, 3, 2, 1}`

Output: `6`

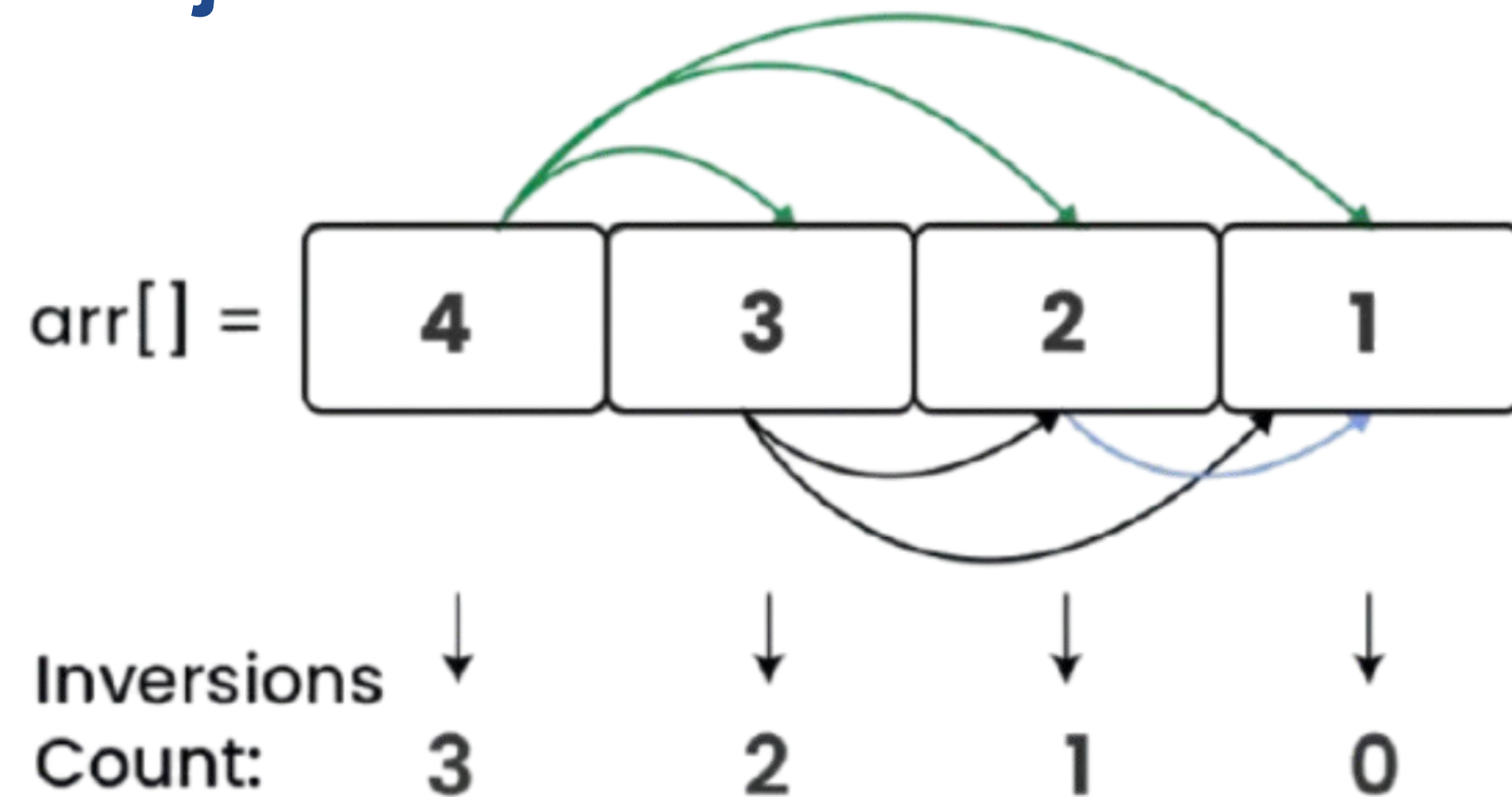
Output: 0

- Explanation: There are no pairs of indices (i, j) in the given array that satisfy the condition $arr[i] > arr[j]$ while maintaining $i < j$.

Input: `arr = {10, 10, 10}`

Output: 0

Inversion Count: `arr[i] > arr[j]` such that $i < j$.



Total Inversions Count: $3 + 2 + 1 + 0 = 6$

Quick Sort: An Efficient Sorting Algorithm

Quick Sort is a rapid, in-place divide-and-conquer algorithm designed for effectively sorting elements by following these steps:

- Dividing the list by selecting a pivot element.
- Partitioning: Rearranging elements so that all smaller values are positioned before the pivot and larger ones follow.
- Recursively sorting the subarrays on either side of the pivot until the base case (single-element arrays) is reached.

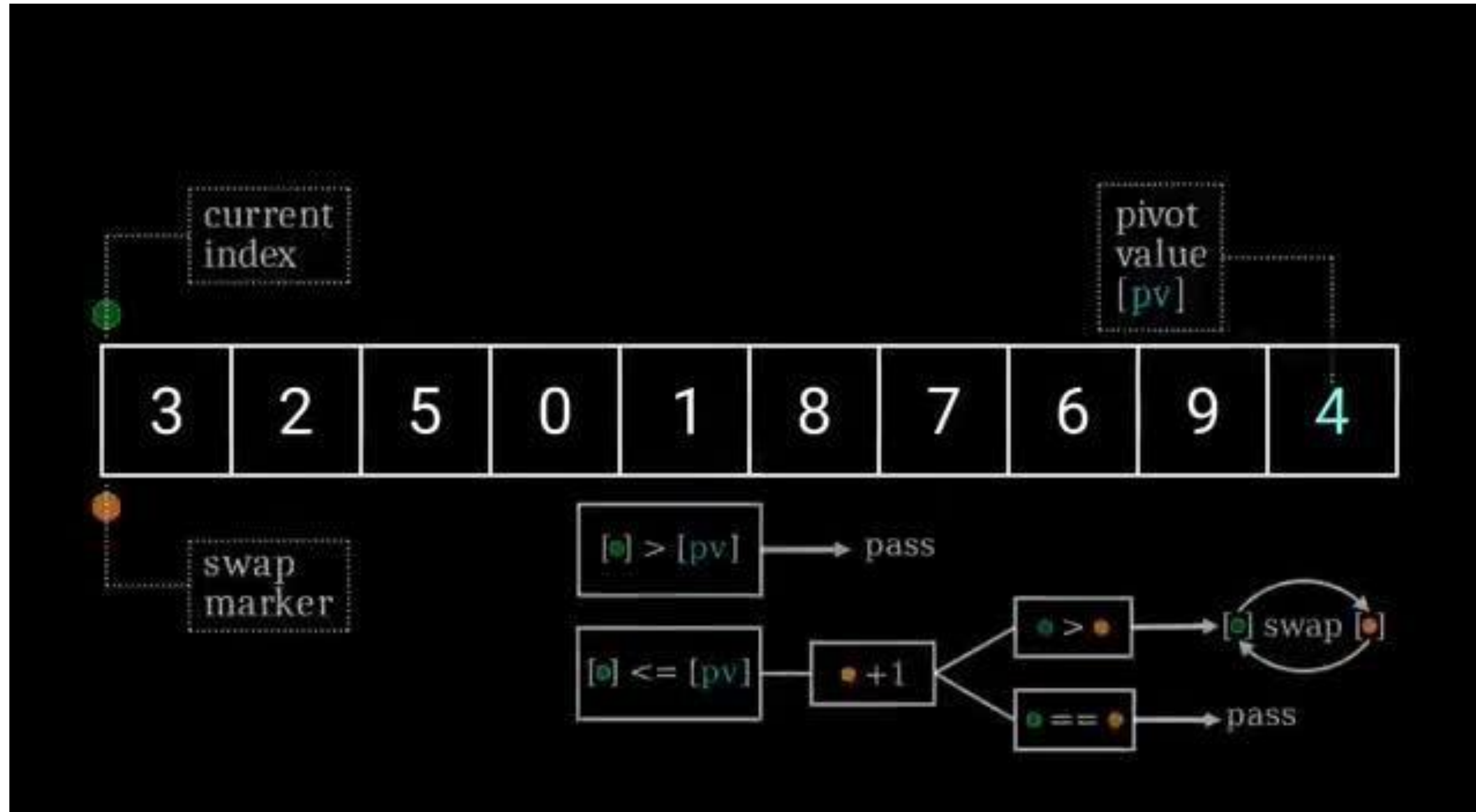
Each step progressively narrows down the sorting region, leading to an efficiently arranged array.

Key Features:

- Time Complexity:
 - Best & Average: $O(n \log n)$
 - Worst Case: $O(n^2)$ (occurs with poor pivot selection, such as pre-sorted arrays without randomization)
- Space Complexity:
 - $O(\log n)$ (due to the recursive call stack)
- In-place sorting: No additional array is necessary.
- Stability:
 -  Not stable (equal elements may not keep their original order)

Used When:

- Speed is essential, particularly for large datasets.
- Memory usage needs to be kept to a minimum.



code for quick sort

```
1  #include <iostream>
2  using namespace std;
3
4  int partition(int arr[], int low, int high) {
5      int pivot = arr[high];
6      int i = low - 1;
7      for (int j = low; j < high; j++)
8          if (arr[j] < pivot)
9              swap(arr[++i], arr[j]);
10     swap(arr[i + 1], arr[high]);
11     return i + 1;
12 }
13
14 void quickSort(int arr[], int low, int high) {
15     if (low < high) {
16         int p = partition(arr, low, high);
17         quickSort(arr, low, p - 1);
18         quickSort(arr, p + 1, high);
19     }
20 }
```

🔧 What This Code Does:

partition() Function:

- Selects the last element as the pivot.
- Positions elements smaller than the pivot to the left and larger elements to the right.
- Returns the appropriate index of the pivot in the sorted array.

quickSort() Function:

- Recursively sorts the array:
 - The left section (before the pivot)
 - The right section (after the pivot)

Main Logic:

- Continues the process of partitioning and sorting until the entire array is arranged in order.



Finding the Kth Largest Element in an Array

Given an array called `nums` and an integer `k`, your task is to return the `k`th largest element from the array.

Explanation

This element corresponds to the value you would find at index `k-1` if the array were sorted in descending order.

Examples:

- Input: `nums = [7, 10, 4, 3, 20, 15]`, `k = 3`
- Output: 10
- Input: `nums = [1]`, `k = 1`
- Output: 1
- Input: `nums = [3, 2, 1, 5, 6, 4]`, `k = 2`
- Output: 5



Practice Problems:

Sort Colors (Dutch National Flag Problem)

Sort an array containing 0s, 1s, and 2s without utilizing any sorting libraries.

Input: `nums = [2, 0, 2, 1, 1, 0]`

Output: `[0, 0, 1, 1, 2, 2]`

Merging Two Sorted Arrays

Combine two sorted arrays into a single sorted array (in-place).

Input:

`nums1 = [1, 2, 3, 0, 0, 0]`

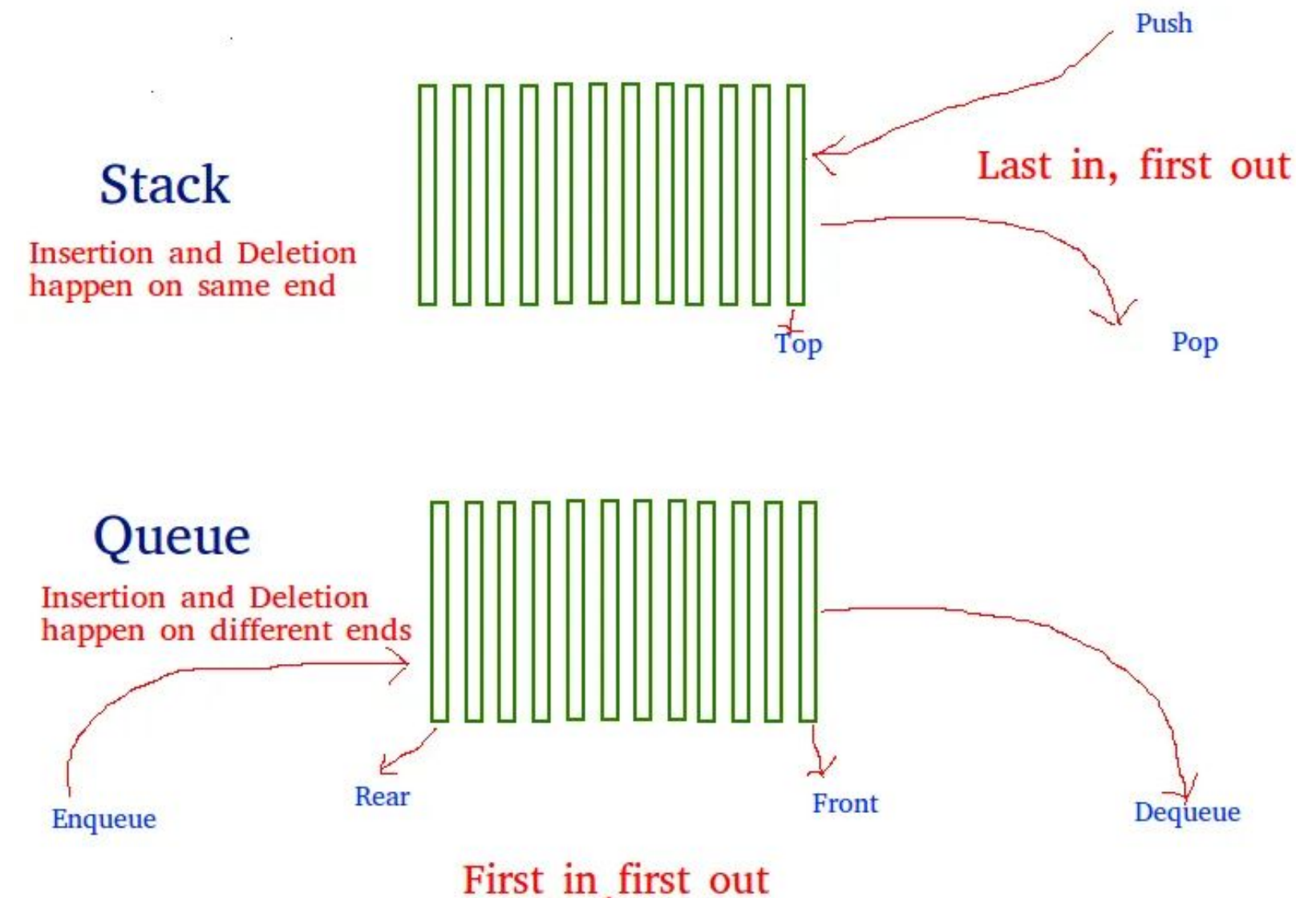
`nums2 = [2, 5, 6]`

Output: `[1, 2, 2, 3, 5, 6]`

Try to solve both questions within a Θ (`nums.size()`) and Θ (`nums1.size() + nums2.size()`).

Introduction to Stacks and Queues

- Both stacks and queues are linear data structures, but they differ in how they handle the insertion and removal of elements.
- Understanding the differences and when to use each is crucial for solving many programming problems efficiently.



What is a Stack?

A stack is a data structure that follows the LIFO (Last In, First Out) principle.

This means that the last element added to the stack is the first one to be removed. Think of a stack like a pile of books: you can only take the book from

Basic Operations

1. Push: Adds an element to the top of the stack.
2. Pop: Removes the top element.
3. Top/Ppeek: Accesses the top element without removing it.
4. isEmpty: Checks if the stack is empty.



Stack Example in C++ Using Arrays

```
1  #include <iostream>
2  using namespace std;
3
4  class Stack {
5      int *arr, top, capacity;
6  public:
7      Stack(int size) : arr(new int[size]), top(-1), capacity(size) {}
8      void push(int val) {
9          if (top == capacity - 1) cout << "Overflow\n";
10         else arr[++top] = val;
11     }
12     int pop() {
13         return (top == -1) ? (cout << "Underflow\n", -1) : arr[top--];
14     }
15     int peek() {
16         return (top == -1) ? (cout << "Empty\n", -1) : arr[top];
17     }
18     bool isEmpty() { return top == -1; }
19 };
20 int main() {
21     Stack s(5);
22     s.push(10); s.push(20);
23     cout << "Top: " << s.peek() << '\n';
24     s.pop();
25     cout << "Top after pop: " << s.peek() << '\n';
26 }
```

Stack in STL

```
1  #include <iostream>
2  #include <stack>
3  using namespace std;
4  int main() {
5      stack<int> s;
6      s.push(10);
7      s.push(20);
8      cout << "Top element is " << s.top() << endl;
9      s.pop();
10     cout << "Top element after pop is " << s.top() << endl;
11     return 0;
12 }
```

Where to Use Stacks?

Recursion: Many recursive problems (like depth-first search) use stacks

Backtracking: Stacks help in undo operations (e.g., navigating a web browser or history)

Expression Evaluation: Converting or evaluating expressions in postfix, prefix, or infix notation often requires stacks.

Brackets Matching: Checking if a sequence of brackets is balanced is another common use case for stacks.

When to Use a Stack in a Problem?

If the problem requires processing data in reverse order or needs to track recent actions (like undo operations), a stack is likely the right choice.

Problems :

1. Reversing Data For Example: $\{1, 2, 3, 4\} \rightarrow \{4,$

2. Convert Decimal to $\{3, 2, 1\}$

3. Given an array of integers, for each element, find the next greater element to its right. If there is no greater element, output is right. Input : $[4, 5, 2, 25]$ Output : $[5, 25,$

4. Given an array of integers heights representing the heights of bars in a histogram, find the area of the largest rectangle that can be formed inside the histogram.

Input: $[6, 2, 5, 4, 5, 1, 6]$

Output: 12



What is a Queue?

A queue follows the FIFO (First In, First Out) principle. This means that the first element added to the queue is the first one to be removed, like a line of

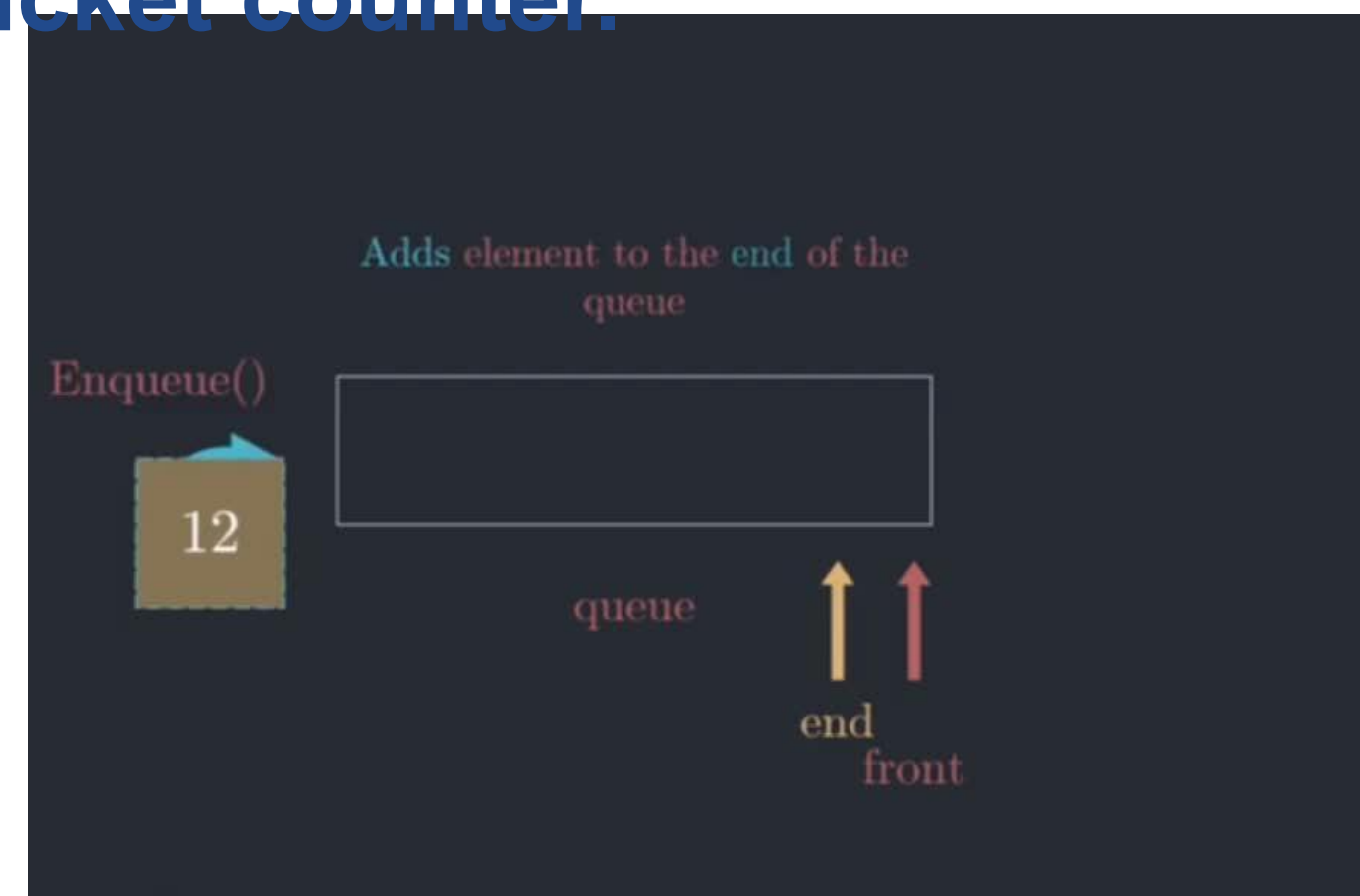
Basic Operations people waiting at a ticket counter.

1. Enqueue: Adds an element to the end of the queue.

2. Dequeue: Removes the front element.

3. Front: Accesses the front element.

4. isEmpty: Checks if the queue is



Queue Example in C++ Using Arrays

```
1  #include <iostream>
2  using namespace std;
3
4  class Queue {
5      int *arr, front, rear, capacity;
6  public:
7      Queue(int cap) : front(0), rear(0), capacity(cap) { arr = new int[capacity]; }
8      ~Queue() { delete[] arr; }
9      bool isEmpty() { return front == rear; }
10     bool isFull() { return rear - front == capacity; }
11     void enqueue(int x) {
12         if (isFull()) { cout << "Queue Full\n";
13             return; }
14         arr[rear % capacity] = x;
15         rear++;
16     }
17     int dequeue() {
18         if (isEmpty()) { cout << "Queue Empty\n";
19             return -1; }
20         return arr[front++ % capacity];
21     }
22     int getFront() {
23         if (isEmpty()) { cout << "Queue Empty\n";
24             return -1; }
25         return arr[front % capacity];
26     }
27 };
28
29 int main() {
30     Queue q(5);
31     q.enqueue(10);
32     q.enqueue(20);
33     cout << "Front: " << q.getFront() << endl;
34     q.dequeue();
35     cout << "Front after dequeue: " << q.getFront() << endl;
36 }
```

Queue in STL

```
1  #include <iostream>
2  #include <queue>
3  using namespace std;
4  int main() {
5      queue<int> q;
6      q.push(10);
7      q.push(20);
8      cout << "Front element is " << q.front() << endl;
9      q.pop();
10     cout << "Front element after dequeue is " << q.front() << endl;
11     return 0;
12 }
```




Identifying Whether to Use a Stack or Queue ?

Use a stack if the problem involves:

Reversing data.

Backtracking or undoing operations.

Matching elements (like parentheses or

tags)
Use a queue if the problem involves:

Processing data in the order it arrives.

Task scheduling or BFS-like algorithms.

**Systems where tasks need to be handled in
sequence**



Valid Parentheses

Given a string `s` containing just the characters `'('`, `')'`, `'{'`, `'}'`, `'['` and `']'`, determine if the input string is valid.

An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

```
Example 1:  
Input: s = "()"   
Output: true  
Example 2:  
Input: s = "()[]{}"   
Output: true  
Example 3:  
Input: s = "]"   
Output: false  
Example 4:  
Input: s = "([])"   
Output: true
```



Valid Parentheses

Given a string `s` containing just the characters `'('`, `')'`, `'{'`, `'}'`, `'['` and `']'`, determine if the input string is valid.

An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

Example 1:
Input: `s = "()"`
Output: `true`
Example 2:
Input: `s = "()[]{}"`
Output: `true`
Example 3:
Input: `s = "([]"`
Output: `false`
Example 4:
Input: `s = "([)]"`
Output: `true`

Solution

```
1  class Solution {
2  public:
3      bool isValid(string s) {
4          stack<char> st; //create an empty stack to store opening brackets
5          for(char c : s){
6              if(c == '(' || c == '{' || c == '['){
7                  st.push(c);
8              }
9              else{
10                 if(st.empty() ||
11                    (c == ')' && st.top() != '(') ||
12                    (c == '}' && st.top() != '{') ||
13                    (c == ']' && st.top() != '[')){
14                     return false;
15                 }
16                 st.pop();
17             }
18         }
19         return st.empty();
20     }
21 };
22
23 // Time complexity : O(n)
24 // Space Complexity : O(n)
```


Evaluate Reverse Polish Notation

You are given an array of strings `tokens` that represents an arithmetic expression in a Reverse Polish Notation.

Evaluate the expression. Return *an integer that represents the value of the expression*.

Note that:

- The valid operators are `'+'`, `'-'`, `'*'`, and `'/'`.
- Each operand may be an integer or another expression.
- The division between two integers always **truncates toward zero**.
- There will not be any division by zero.
- The input represents a valid arithmetic expression in a reverse polish notation.
- The answer and all the intermediate calculations can be represented in a 32-bit integer.

Example 1:

Input: `tokens = ["2","1","+","3","*"]`

Output: 9

Explanation: $((2 + 1) * 3) = 9$

Example 2:

Input: `tokens = ["4","13","5","/","+"]`

Output: 6

Explanation: $(4 + (13 / 5)) = 6$

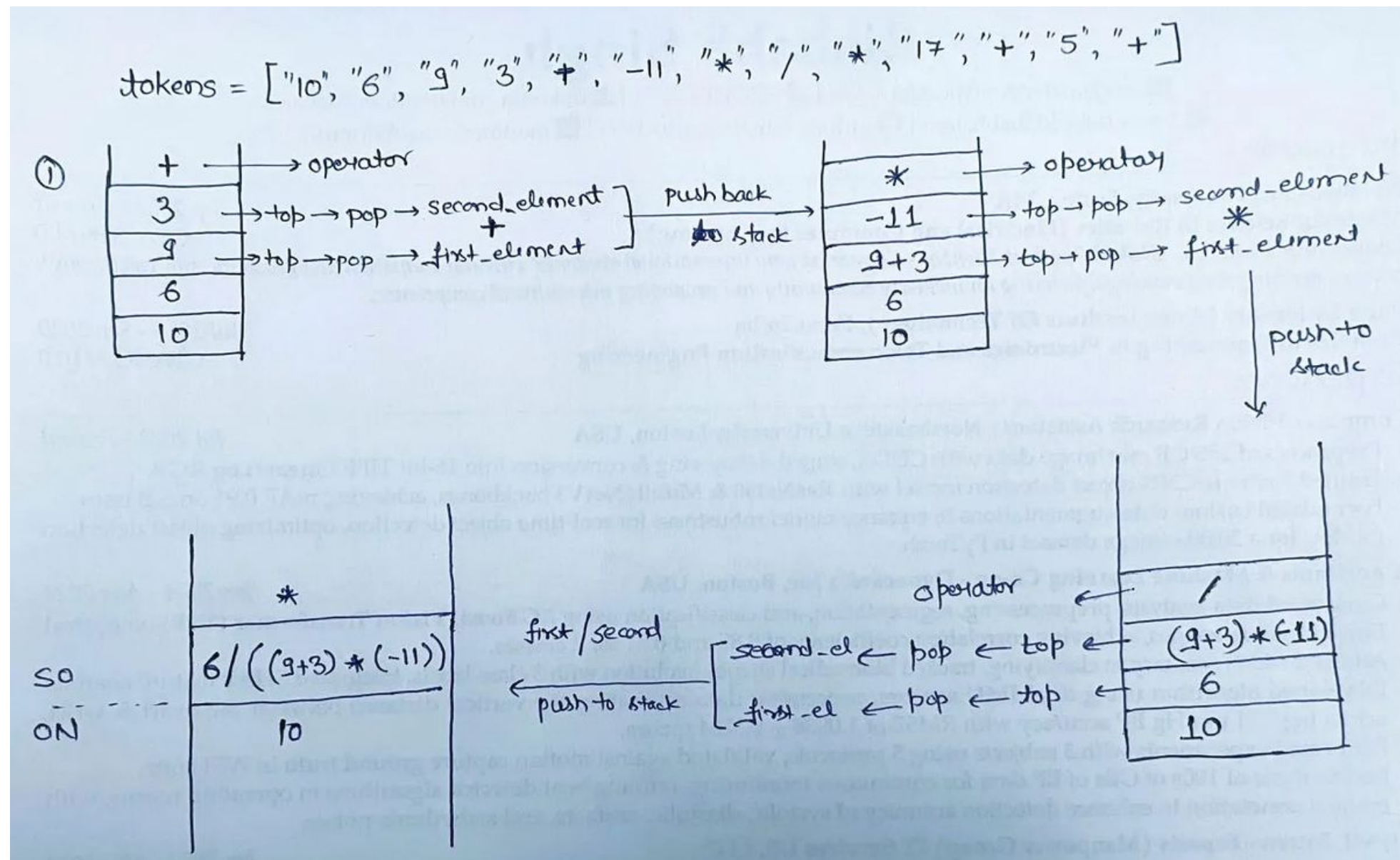
Example 3:

Input: `tokens = ["10","6","9","3","+","-11","*","/","*", "17","+","5","+"]`

Output: 22

Explanation: $((10 * (6 / ((9 + 3) * -11))) + 17) + 5$
 $= ((10 * (6 / (12 * -11))) + 17) + 5$
 $= ((10 * (6 / -132)) + 17) + 5$
 $= ((10 * 0) + 17) + 5$
 $= (0 + 17) + 5$
 $= 17 + 5$
 $= 22$

Explanation

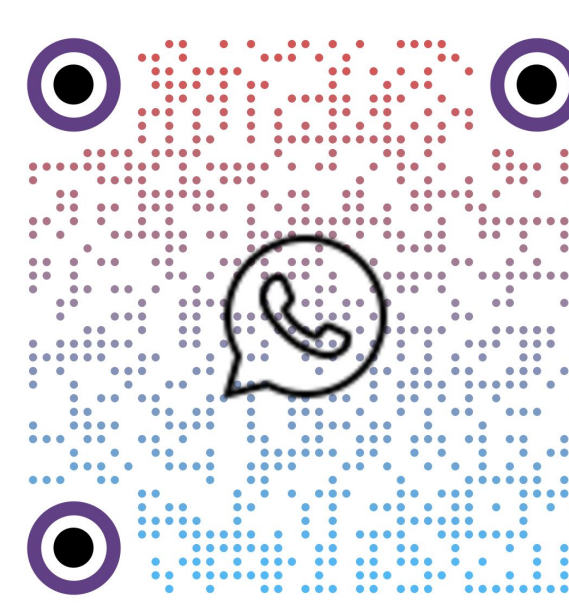
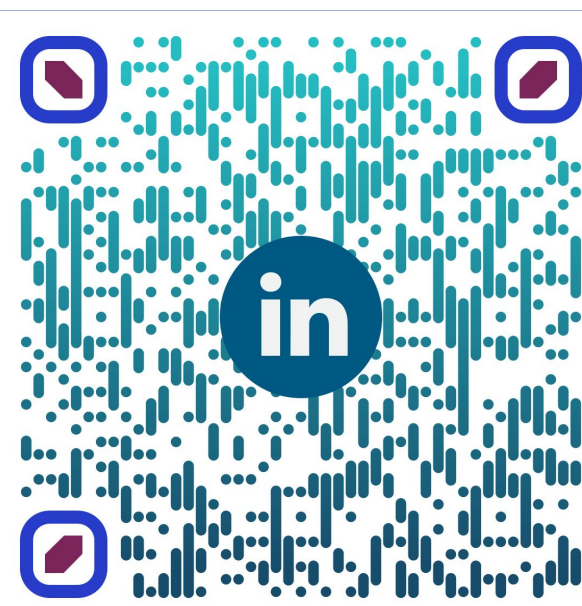
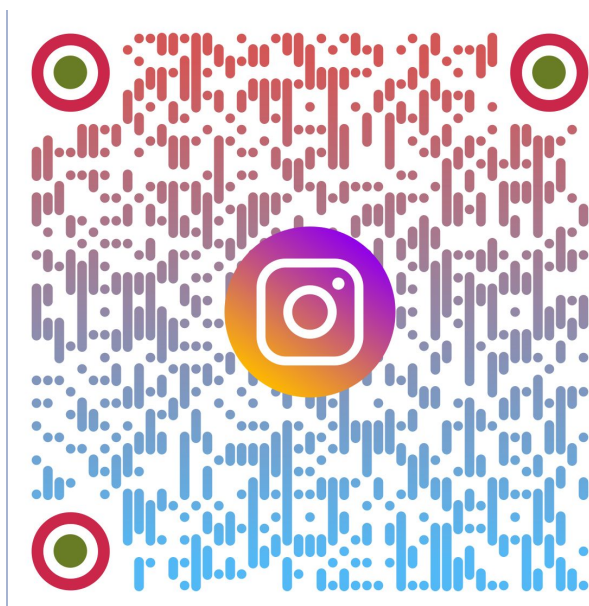


```

1  #include <stack>
2  using namespace std;
3  class Solution {
4  public:
5      int tung_tung_tung_sahur(vector<string>& tokens) {
6          stack<int> st;
7          for (const string& c : tokens) {
8              if (c == "+" || c == "-" || c == "*" || c == "/") {
9                  int b = st.top(); st.pop();
10                 int a = st.top(); st.pop();
11                 if (c == "+") st.push(a + b);
12                 else if (c == "-") st.push(a - b);
13                 else if (c == "*") st.push(a * b);
14                 else st.push(a / b);
15             } else {
16                 st.push(stoi(c));
17             }
18         }
19         return st.top();
20     }
21 };

```


Connect with us



Thank You

For your time, patience, and
attention!



Feedback form

